

Entrega Inicial de Bootcamp — API de Gerenciamento de Tarefas

Sumário

1. *Identificação da entrega*
2. *Problema escolhido e objetivo*
3. *Tecnologias e ambiente*
4. *Arquitetura e organização*
5. *Especificação da API*
6. *Processo de desenvolvimento no GitHub*
7. *Uso do agente de IA*
8. *Testes, Docker e integração contínua*
9. *Evidências de validação*
10. *Divisão de responsabilidades*
11. *Conclusão*
12. *Referências*

1. Identificação da entrega

Este relatório apresenta a Entrega Inicial do projeto de uma API REST simples para gerenciamento de tarefas. O desenvolvimento foi organizado por meio de Issues, branches, Pull Requests, revisão por pares, integração contínua e documentação técnica.

1.1 Equipe

Integrante	Responsabilidade	RA
Vitor de Assis Patricio Borges	Especificação e decisões técnicas	22304737
Gabriela Vieira Baptista	GitHub, agente de IA e documentação final	22510133
Felipe Domingos Ribeiro Pereira	Implementação e banco de dados	22604250
Caike Ribeiro Menezes	Ambiente, testes e integração contínua	22601978

2. Problema escolhido e objetivo

A equipe escolheu desenvolver uma API de gerenciamento de tarefas. O problema é representar, por meio de operações HTTP simples, o ciclo de vida de uma tarefa: criação, consulta, atualização de status e exclusão.

O objetivo é disponibilizar uma API pequena, compreensível e testável, adequada para demonstrar práticas de engenharia de software, incluindo especificação orientada a requisitos, divisão de trabalho, controle de versão, testes automatizados e integração contínua.

2.1 Escopo funcional

A primeira versão contém quatro operações principais:

Operação	Rota	Finalidade
POST	/tarefas	Criar uma nova tarefa.
GET	/tarefas	Listar tarefas cadastradas.
PATCH	/tarefas/{id}	Atualizar dados ou concluir uma tarefa.
DELETE	/tarefas/{id}	Excluir uma tarefa existente.

Cada tarefa é representada pelos campos `id`, `titulo` e `concluida`. O sistema também prevê respostas de validação para entradas inválidas e para identificadores inexistentes.

3. Tecnologias e ambiente

A solução foi construída em Python, utilizando FastAPI para exposição das rotas HTTP, Uvicorn para execução local, SQLite como banco de dados, SQLAlchemy para persistência e Pydantic para validação dos dados. A suíte automatizada utiliza pytest; Docker e GitHub Actions apoiam a reprodução e a integração contínua.

Tecnologia	Uso no projeto
Python 3.12	Linguagem principal da aplicação.
FastAPI	Framework para construção da API REST.
Uvicorn	Servidor ASGI para execução local.
SQLite	Banco de dados leve para persistência.
SQLAlchemy	Mapeamento e acesso aos dados.
Pydantic	Validação e serialização dos dados.
pytest	Testes automatizados.
Docker	Padronização do ambiente de execução.
GitHub Actions	Execução automática dos testes.

4. Arquitetura e organização

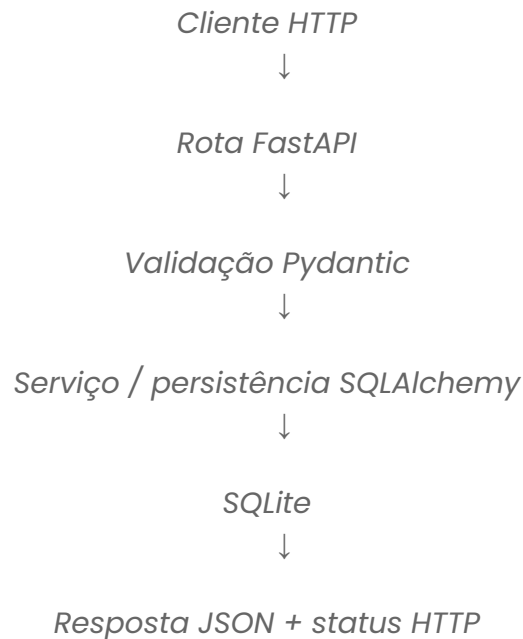
A aplicação foi organizada em camadas simples, separando o ponto de entrada da API, o acesso ao banco, os modelos, os schemas de validação e os testes. Essa separação reduz o acoplamento e permite que alterações de implementação sejam verificadas por testes independentes.

A estrutura integrada na branch de desenvolvimento contém os seguintes elementos:

- `.github/workflows/` # Integração contínua
- `app/` # Código da API e persistência
- `docs/` # Especificação SDD e decisões técnicas
- `evidencias/` # Prints e registros da entrega
- `scripts/` # Scripts auxiliares
- `tests/` # Testes automatizados
- `AGENTS.md` # Diretrizes para uso do agente de IA
- `Dockerfile` # Ambiente containerizado
- `pyproject.toml` # Dependências e configuração
- `README.md` # Documentação principal

4.1 Fluxo de uma requisição

A requisição chega à rota FastAPI, é validada pelo schema correspondente, processada pela camada de serviço e persistida no SQLite. A resposta é serializada no formato JSON e devolvida com o código HTTP adequado.



5. Especificação da API

A especificação SDD definiu os contratos, as regras de negócio, a arquitetura e os critérios de aceitação antes da integração da implementação. Os contratos foram revisados no fluxo de Pull Request e serviram como referência para o trabalho de Felipe e Caike.

5.1 Contratos principais

A criação recebe um título válido e inicia a tarefa como não concluída. A listagem retorna as tarefas persistidas. A atualização permite alterar o título e/ou concluir a tarefa. A exclusão remove o registro e retorna **204 No Content** quando a operação é realizada com sucesso.

Os principais códigos utilizados são:

Código	Significado
201	Tarefa criada com sucesso.
200	Consulta ou atualização concluída.
204	Tarefa excluída sem conteúdo de resposta.
404	Tarefa não encontrada.
422	Dados de entrada inválidos.

6. Processo de desenvolvimento no GitHub

O trabalho foi dividido em Issues e branches específicas. A branch **main** representa a versão principal, enquanto **develop** funciona como branch de integração. As alterações foram desenvolvidas em branches de funcionalidade e documentação, submetidas por Pull Request e revisadas antes do merge.

O fluxo utilizado foi:

Issue → branch → commit → Pull Request → revisão → check → merge em develop

A equipe configurou um GitHub Project com os status **Backlog**, **Em andamento**, **Em revisão**, **Bloqueado** e **Concluído**. A proteção da branch principal foi planejada para impedir alterações diretas sem revisão.

Os Pull Requests registraram, entre outras etapas, a criação das instruções do agente, a especificação SDD, o README, a implementação da API e a configuração de testes, Docker e GitHub Actions.

7. Uso do agente de IA

O arquivo **AGENTS.md** registra o contexto do projeto e as regras para uso do agente de IA. O documento orienta o agente sobre a stack, as rotas, o modelo de dados, as regras de negócio e os comandos de instalação e testes.

A equipe adotou como regra que alterações de comportamento devem ser acompanhadas de testes, que o agente deve consultar a especificação antes de

implementar e que cada resultado gerado com apoio de IA deve ser revisado por integrantes da equipe. O agente foi utilizado como apoio à organização, documentação, elaboração de testes e refinamento das instruções, sem substituir a revisão humana.

8. Testes, Docker e integração contínua

A suíte automatizada cobre o fluxo de criação, listagem, atualização e exclusão, além de entradas inválidas e identificadores inexistentes. O projeto pode ser executado localmente com `pytest` e também em container Docker.

Os comandos documentados são:

```
python -m pip install -e ".[test]"

python -m pytest -v

python -m uvicorn app.main:app --reload
```

Para o container:

```
docker build -t tarefas-api:test .

docker run --rm tarefas-api:test python -m pytest -v
```

O workflow em [.github/workflows/tests.yml](#) instala as dependências e executa os testes automaticamente em alterações submetidas ao repositório. As evidências do GitHub Actions registram execuções concluídas com sucesso.

A suíte automatizada foi executada com sucesso, totalizando **80** testes aprovados. Os testes cobrem o fluxo de criação, listagem, atualização e exclusão de tarefas, além do tratamento de dados inválidos e de identificadores inexistentes.

9. Evidências de validação

As evidências visuais da entrega foram organizadas na pasta [evidencias/](#) do repositório. Para evitar que o relatório fique excessivamente extenso, as imagens

não foram inseridas individualmente neste PDF. Elas podem ser consultadas diretamente no seguinte endereço:

<https://github.com/gabrielavieirab/tarefas-api/tree/docs/evidencias-finais/evidencias>

As evidências disponíveis incluem a organização do GitHub Project, o histórico de Pull Requests, as branches, a execução do GitHub Actions, a documentação Swagger e os testes funcionais da API.

Relação das evidências

1. **Pull Requests:** comprova o fluxo de desenvolvimento, revisão e integração das alterações.
2. **Swagger:** comprova a disponibilização das quatro rotas principais da API.
3. **POST /tarefas:** comprova a criação de uma tarefa com resposta HTTP 201.
4. **GET /tarefas:** comprova a listagem de tarefas com resposta HTTP 200.
5. **PATCH /tarefas/{id}:** comprova a atualização ou conclusão de uma tarefa.
6. **Resposta do PATCH:** comprova a atualização realizada com resposta HTTP 200.
7. **DELETE /tarefas/{id}:** comprova a exclusão de uma tarefa com resposta HTTP 204.
8. **GET após exclusão:** comprova que a tarefa excluída não aparece mais na listagem.
9. **Erro 422:** comprova o tratamento de dados de entrada inválidos.
10. **Erro 404:** comprova o tratamento de identificadores inexistentes.
11. **GitHub Actions:** comprova a execução automatizada da suíte de testes.

10. Divisão de responsabilidades

Vitor foi responsável pela especificação SDD, pelos contratos e pelas decisões técnicas. Gabriela organizou o repositório, as Issues, o Project, o fluxo de Pull Requests, o agente de IA, o README e as evidências. Felipe implementou o banco de dados e as rotas da API. Caike preparou o ambiente, os testes, o Docker e a integração contínua.

Essa divisão permitiu que a equipe trabalhasse em paralelo sem perder a rastreabilidade entre requisito, implementação, teste e evidência.

11. Conclusão

A equipe estruturou e documentou uma API funcional de gerenciamento de tarefas, com quatro operações REST, persistência em SQLite, validação de dados, testes automatizados, execução em Docker e integração contínua. O uso de Issues, branches e Pull Requests tornou as responsabilidades e as decisões técnicas rastreáveis.

A entrega também reúne evidências do processo e da validação manual. Antes da submissão final, recomenda-se confirmar que o PR de documentação foi revisado e integrado, que todas as Issues correspondentes estão atualizadas no Project e que a versão final do código foi validada na branch de integração.

12. Referências

- [*Repositório da equipe — tarefas-api.*](#)
- [*FastAPI — documentação oficial.*](#)
- [*pytest — documentação oficial.*](#)
- [*GitHub Actions — documentação oficial.*](#)
- [*Docker — documentação oficial.*](#)